Eulers metode for annenordens differensialligning

Mål

Målet med oppgaven er å forstå hvordan vi kan løse en *annenordens differensialligning* numerisk ved å skrive den om til et system av to førsteordens ligninger og bruke Eulers metode.

Teori: Dempede svingninger

Vi ser på den klassiske ligningen for en dempet harmonisk oscillator:

$$F = -kx - bv \tag{1}$$

Denne kan f.eks. beskrive en pendel som sakte stoppes av luftmotstand. kx er kraften som trekker pendelen mot midten ($x = 0$), og bv er luftmotstand. v er selvsagt hastigheten, og vi legger merke til at luftmotstand øker når hastigheten øker.

Siden $F = ma$, $a = \frac{d^2x}{dt^2}$ og $v = \frac{dx}{dt}$ kan kraftligningen skrives som differensialligningen

$$\frac{d^2x}{dt^2} = -\alpha\frac{dx}{dt} - \beta x, \tag{2}$$

der $\alpha = b/m$ og $\beta = k/m$.

Dette er en annenordens differensialligningligning siden den involverer den andrederiverte. For å bruke Eulers metode må vi skrive den om til to førsteordens ligninger.

Vi gjeninnfører

$$v = \frac{dx}{dt},$$

og omskriver differensialligningen (2) som:

$$\frac{dx}{dt} = v \iff dx = v\,dt \tag{3a}$$

$$\frac{dv}{dt} = -\alpha v - \beta x \iff dv = -(\alpha v + \beta x)\,dt \tag{3b}$$

Nå kan vi bruke Eulers metode på begge ligningene samtidig:

$$
\begin{aligned}
t_{n+1} &= t_n + dt, && && \text{(4a)}\\
x_{n+1} &= x_n + dx && && \text{(4b)}\\
\implies x_{n+1} &= x_n + v_n\,dt && \text{(fra (3a))}, && \text{(4c)}\\
v_{n+1} &= v_n + dv && && \text{(4d)}\\
\implies v_{n+1} &= v_n - (\alpha v_n + \beta x_n)\,dt && \text{(fra (3b))}. && \text{(4e)}
\end{aligned}
$$

Oppgave 1: Forstå systemet

La parameterne være gitt ved

$$\alpha = 0.1, \quad \beta = 1$$

og la initialbetingelsene være

$$x(0) = 1, \quad v(0) = 0.$$

Med andre ord har vi $(t_0, v_0, x_0) = (0, 0, 1)$.

Bruk $dt = 0.1$ og regn ut følgende.

a) Regn ut t_1, v_1 og x_1.[1]

b) Regn ut x_2 og v_2.[2]

c) Fortsett til du går lei.

[1] $(t_1, v_1, x_1) = (0.1, -0.1, 1)$.
[2] $(t_2, v_2, x_2) = (0.2, -0.195, 0.99)$.

Oppgave 2: Programmer funksjonen

Programmer funksjonen og definer parameterne:

```
x0 = 1
v0 = 0
alpha = 0.5
beta = 4

t0 = 0
t_slutt = 10
dt = 0.1

def f(x, v):
    return -alpha*v - beta*x
```

Oppgave 3: Implementer Eulers metode

```
x = x0
v = v0
t = t0

while t <= t_slutt:
    x = x + v * dt
    v = v + f(x, v) * dt
    t = t + dt
    print(t, v, x)
```

a) Hvilken linje tilsvarer formelen for v_{n+1}?

b) Sjekk at utregningene fra oppgave 1 stemmer.

Oppgave 5: Tegn grafen

```
import matplotlib.pyplot as plt

x_values = [x0]
v_values = [v0]
t_values = [t0]

x = x0
```

```
v = v0
t = t0

while t <= t_slutt:
    t = t + dt
    x = x + v * dt
    v = v + f(x, v) * dt


    x_values.append(x)
    v_values.append(v)
    t_values.append(t)

plt.plot(t_values, x_values)
plt.xlabel("Tid")
plt.ylabel("Posisjon")
plt.title("Dempede svingninger med Eulers metode")
plt.grid()
plt.show()
```

Oppgave 6: Sammenlign med eksakt løsning

Differensiallignigen har tre forskjellige løsninger. Den første for $\alpha^2 < 4\beta$:

$$x(t) = \left[x_0 \cos\left(\frac{\sqrt{4\beta - \alpha^2}}{2}\, t\right) + \frac{2v_0 + \alpha x_0}{\sqrt{4\beta - \alpha^2}} \sin\left(\frac{\sqrt{4\beta - \alpha^2}}{2}\, t\right)\right] e^{-\frac{\alpha}{2}t} \tag{5}$$

Den andre for $\alpha^2 = 4\beta$:

$$x(t) = \left[x_0 + \left(v_0 + \frac{\alpha}{2} x_0\right) t\right] e^{-\frac{\alpha}{2}t} \tag{6}$$

Og den tredje for $\alpha^2 > 4\beta$:

$$x(t) = \frac{v_0 - r_- x_0}{r_+ - r_-}\, e^{r_+ t} + \frac{r_+ x_0 - v_0}{r_+ - r_-}\, e^{r_- t}, \tag{7}$$

der

$$r_\pm = \frac{-\alpha \pm \sqrt{\alpha^2 - 4\beta}}{2}.$$

I Python ser dette slik ut:

```
import numpy as np

def X(t):
```

```
    D = alpha**2 - 4*beta

    # Underdempet (komplekse røtter)
    if D < 0:
        omega = np.sqrt(4*beta - alpha**2) / 2
        C1 = x0
        C2 = (v0 + (alpha/2)*x0) / omega
        return np.exp(-alpha*t/2) * (
            C1*np.cos(omega*t) + C2*np.sin(omega*t)
        )

    # Kritisk dempet (dobbel rot)
    elif D == 0:
        r = -alpha/2
        C1 = x0
        C2 = v0 - r*x0
        return (C1 + C2*t) * np.exp(r*t)

    # Overdempet (to reelle røtter)
    else:
        r1 = (-alpha + np.sqrt(D)) / 2
        r2 = (-alpha - np.sqrt(D)) / 2

        C2 = (v0 - r1*x0) / (r2 - r1)
        C1 = x0 - C2

        return C1*np.exp(r1*t) + C2*np.exp(r2*t)

T = np.linspace(t0, t_slutt, 500)

plt.plot(T, X(T), "--")
```